📌 Handling Missing Values in SQL – Simple Notes

🔷 What Are Missing Values?

Missing values are data entries stored as **NULL** in SQL.

They can:

- Reduce accuracy
- Cause calculation errors
- Affect analysis and reporting

🗃 Sample Table: messy_indian_dataset

The table contains customer details like:

- id
- name
- age
- gender
- email
- phone_number
- city
- state
- purchase_amount
- purchase_date

Some rows contain **NULL values** in:

- age
- email
- phone_number
- purchase_amount
- purchase_date

🔎 1️⃣ Finding Missing Values

To find rows where **any column has NULL**:

```
SELECT * FROM messy_indian_dataset
WHERE name IS NULL
```

```
  OR age IS NULL
  OR gender IS NULL
  OR email IS NULL
  OR city IS NULL
  OR phone_number IS NULL
  OR state IS NULL
  OR purchase_amount IS NULL
  OR purchase_date IS NULL;
```

👉 IS NULL is used to detect missing values.
👉 Returns incomplete records.

✅ 2️⃣ Finding Rows Without Missing Values

To get only **complete records**:

```
SELECT * FROM messy_indian_dataset
WHERE name IS NOT NULL
  AND age IS NOT NULL
  AND gender IS NOT NULL
  AND email IS NOT NULL
  AND city IS NOT NULL
  AND phone_number IS NOT NULL
  AND state IS NOT NULL
  AND purchase_amount IS NOT NULL
  AND purchase_date IS NOT NULL;
```

👉 IS NOT NULL ensures all values are present.

💾 3️⃣ Saving Clean Data in New Table

```
CREATE TABLE clean_table AS
SELECT * FROM messy_indian_dataset
WHERE name IS NOT NULL
  AND age IS NOT NULL
  AND gender IS NOT NULL
```

```
AND email IS NOT NULL
AND city IS NOT NULL
AND phone_number IS NOT NULL
AND state IS NOT NULL
AND purchase_amount IS NOT NULL
AND purchase_date IS NOT NULL;
```

👉 Creates a new table with only clean records.

🔄 4️⃣ Filling Missing Values

🔷 Using COALESCE()

COALESCE(column, value)
Returns the column value if NOT NULL, otherwise returns the replacement value.

👧 Filling Missing Age

```
UPDATE messy_indian_dataset
SET age = COALESCE(age, 0);
```

👉 Replaces NULL age with 0.

📝 Filling Other NULL Values

```
UPDATE messy_indian_dataset
SET
  name = COALESCE(name, 'Unknown'),
  gender = COALESCE(gender, 'Unknown'),
  email = COALESCE(email, 'Unknown'),
  phone_number = COALESCE(phone_number, 'Unknown'),
  state = COALESCE(state, 'Unknown'),
  purchase_date = COALESCE(purchase_date, '2023-01-01');
```

👉 Replaces NULL values with default text or date.

📊 5️⃣ Filling Missing Values with Calculated Values

Instead of fixed values, we can use statistics.

💰 Filling NULL Purchase Amount with Average

```
UPDATE messy_indian_dataset
SET purchase_amount = (
  SELECT AVG(purchase_amount)
  FROM messy_indian_dataset
)
WHERE purchase_amount IS NULL;
```

👉 Replaces NULL with average purchase amount.
👉 Useful for numeric columns.

🏙 Filling NULL City with Most Frequent City

```
UPDATE messy_indian_dataset
SET city = (
  SELECT city FROM (
    SELECT city, COUNT(*) AS frequency
    FROM messy_indian_dataset
    WHERE city IS NOT NULL
    GROUP BY city
    ORDER BY frequency DESC
    LIMIT 1
  ) AS subquery
)
WHERE city IS NULL;
```

👉 Replaces NULL city with the most common city.
👉 Useful for categorical columns.

🧠 Important SQL Functions Used

Function	Purpose
IS NULL	Check missing value

Function	Purpose
IS NOT NULL	Check non-missing value
COALESCE()	Replace NULL with given value
AVG()	Calculate average
COUNT()	Count records
GROUP BY	Group similar values
ORDER BY	Sort results
LIMIT	Restrict number of rows

🎯 Best Practices

✓ Always check NULL values before analysis
✓ Use meaningful replacement values
✓ Use average/median for numeric columns
✓ Use most frequent value for categorical columns
✓ Create backup before updating data

🚀 Final Key Concept

There are **3 main ways to handle missing values in SQL**:

1️⃣ Remove rows with NULL values
2️⃣ Replace NULL with fixed value
3️⃣ Replace NULL with calculated value

Clean data = Accurate analytics 📊